דף סיכום בחינה

מזהה סטודנט: N102215492

מזהה קורס: 20594   שם קורס: מערכות הפעלה

מספר מבחן: 0001

מספר שאלה בטופס בחינה זה	ניקוד מירבי	ציון	התשובה שסומנה	התשובה הנכונה	מיקום השאלה בטופס המקור	מיקום התשובה שסומנה בטופס המקור
1	5.00	5.00	3	2, 3	1	3
2	5.00	5.00	1	1	2	1
3	5.00	5.00	4	4	3	4
4	5.00	5.00	1	1	4	1
5	5.00	5.00	3	3	5	3
6	5.00	5.00	2	2	6	2
7	23.00	23.00	-	-	7	-
8	17.00	17.00	-	-	8	-
9	15.00	15.00	-	-	9	-
10	5.00	5.00	-	-	10	-
11	5.00	5.00	-	-	11	-
12	5.00	5.00	-	-	12	-

כמות תשובות נכונות (עבור שאלות סגורות) : 6

ציון בחינה סופי : 100.00

מקרא צבעים

תשובה שגויה
שאלה שלא נענתה
שאלה מבוטלת

הבחינה הבדוקה בעמודים הבאים

האוניברסיטה
הפתוחה

דף תשובות

מספר גרסה	מספר תעודת זהות	מספר הקורס	תאריך בחינה
0,0,0,1	3,2,6,4,1,8,3,6,5	2,0,5,9,4	0,5,0,3,2,4

0	●	●	●	○
1	○	○	○	●
2	○	○	○	○
3	○	○	○	○
4	○	○	○	○
5	○	○	○	○
6	○	○	○	○
7	○	○	○	○
8	○	○	○	○
9	○	○	○	○

הוראות למילוי דף התשובות

1. נא למלא תשובה אחת בלבד על ידי השחרת העיגול - ●
2. את ההשחרה יש לבצע בעט שחור או כחול בלבד!
3. השימוש בטיפקס או מרקרים אסור!
4. כדי לתקן סימן, יש לסמן X על התשובה הלא נכונה - ⊗
ולהשחיר את התשובה הנכונה - ●
5. כדי לחזור לסימון המקורי, יש לסמן X על התשובה הלא רצויה,
ולהשחיר מחדש את סימן ה- ⊗ באופן שיכסה המחיקה - ⊗

שאלה תשובה	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	שאלה תשובה
א	○	●	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	א
ב	○	○	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ב
ג	●	○	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ג
ד	○	○	●	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ד
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ה
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ו

שאלה תשובה	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	שאלה תשובה
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	א
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ב
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ג
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ד
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ה
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ו

שאלה תשובה	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	שאלה תשובה
א	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	א
ב	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ב
ג	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ג
ד	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ד
ה	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ה
ו	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	ו

שאלון 511 ג 88.3f.2 M3

326418365

מספר תעודת זהות
יש לרשום את כל תשע הספרות

ש מ N102215492
ת.ז: 326418365 מספר סידורי: 16

האוניברסיטה הפתוחה

כ"ה באדר א' תשפ"ד

5
במרץ 2024

מס' שאלון - 511

סמסטר 2024א

20594 / 4

מס' מועד 88

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **11** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלק א' (שאלות אמריקאיות) עליכם לבחור בכל פעם
בתשובה יחידה הנכונה ביותר מבין התשובות.
את התשובות בחלק א' יש לסמן בטופס המחשב המצורף לשאלון.

ג. בחלקים ב' ו ג' מופיעות שאלות פתוחות.
ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ.
אין חובה להשתמש בכל השורות המוקצות לצורך התשובות,
אך אין לחרוג מהמקום המוקצה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.
אין להכניס חומר דיגיטלי!

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

הנחיות למבחן

בחינה בעקרונות מערכות הפעלה

קראו בעיון לפני שתתחילו בפתרון הבחינה!

1. בחלק א׳ (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה הנכונה ביותר מבין התשובות. **את התשובות בחלק א׳ יש לסמן בטופס המחשב המצורף לשאלון.**
2. בחלקים ב׳ ו ג׳ מופיעות שאלות פתוחות. ענו **תשובות מלאות**, **בכתב קריא** ו**בקיצור נמרץ**. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.
3. כל חומר עזר אסור לשימוש בזמן הבחינה פרט למחשבון פשוט
4. משך הבחינה - שלוש שעות.

בהצלחה!

חלק א (30 נקודות)

ענו על שש שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה **ביותר** ולסמן את התשובה **בטופס המחשב המצורף לשאלון.**

שאלה 1

אם ידועים מראש זמני הריצה של כל התהליכים, וניתן לבצע context switch, שיטת התזמון שתוביל לזמן התגובה הממוצע הנמוך ביותר היא:

א. shortest job first <= שיטות ללא preempt, לזמן תגובה מקסימלי

ב. shortest remaining processing time first

[✓ ג.] round robin עם quantum קצר כאשר עבודות חדשות מוכנסות לראש התור <= מכניס חדשים לראש התור ובכך לזמן תגובה קצר

ד. round robin עם quantum קצר כאשר עבודות חדשות מוכנסות לסוף התור

שאלה 2

מה נכון לגבי HYPERVISOR מסוג 1 בתצורת bare metal?

[✓ א.] הקצאת כל משאבי החומרה hardware resources נשלטת **באופן ישיר** ע״י hypervisor. ?

ב. רק הקצאות זיכרון ראשי ופעולות קלט-פלט עוברות דרך hypervisor, אבל הקצאת זמן המעבד CPU הפיזי מבוצעת **ישירות** ע״י מערכת ההפעלה **בלי שיתוף** של hypervisor.

ג. ביצוע כל הפעולות, כולל כלל פעולות ה CPU, עוברות דרך(נשלחות אל CPU לביצוע) ה hypervisor.

ד. כאשר יישום המשתמש מבצע את system calls למערכת הפעלה שרצה על hypervisor, אותה מערכת ההפעלה **לא תנסה** לבצע את ה- system calls ב kernel mode.

לא ברור מה הכוונה. ל-hypervisor יש גישה מלאה ל-hardware.
אבל תהליכים (guest OS) כן יכולים לגשת ישירות לחומרה (המעבד syscalls כמו I/O)

שאלה 3

בחרו מידע אשר **איננו מצוי** ב PCB (process control block) של תהליך :

א. מצב של תהליך (ריצה, המתנה וכו').

ב. מספר תהליך האב של התהליך.

ג. מצביע לטבלת הדפים של התהליך.

[✓ ד.] שיטות הגישה(אקראית או רציפה) לכל הקבצים הפתוחים של התהליך.

שאלה 4

איזו מהבעיות הבאות קיימת כאשר משתמשים ב- user threads ?

[✓ א.] ריבוי חוטים (threads) לא מאפשר ניצול של ריבוי מעבדים

ב. כל הפעולות על חוטים (למשל context switch) סובלות מתקורה של trap

ג. יש צורך בתיווך של מערכת ההפעלה כדי לאפשר תקשורת בין החוטים

ד. אפליקציות לא יכולות לתזמן את החוטים בשיטה האופטימאלית עבורן

המתזמן לא מודע
user thread-ים ... לכן
לא יכול לנצל
ריבוי מעבדים

שאלה 5

אלגוריתם הבנקאי הוא :

א. אלגוריתם המונע deadlock על ידי שבירת האפשרות למעגל בגרף התלויות

ב. אלגוריתם המונע deadlock על ידי הכתבת הסדר ביצוע בקשות של תהליכים שונים

[✓ ג.] אלגוריתם המתחמק מ-deadlock על ידי בדיקה שקיים סדר כלשהו המאפשר הרצה וסיום כל התהליכים הקיימים במערכת

ד. אלגוריתם המאפשר להפוך מצב כלשהו של המערכת למצב בטוח

שאלה 6

מערכת הקבצים של מערכת הפעלה מסוימת משתמשת בשיטת ה I-node עם פרמטרים הבאים :

- גודל הבלוק במערכת הקבצים הוא 2 Kbytes ⇐ 512 מצביעים בבלוק
- כתובת הבלוק היא 4 בתים (bytes)
- 10 שדות של ה I-node יכולים להחזיק ישירות כתובת הבלוק בדיסק ⇐ 10 בלוקים
- שדה נוסף אחד נועד להחזיק כתובת של ה single indirect block 1 בלוק + 512 בלוקים = 513 בלוקים
- עוד שדה נוסף אחד נועד להחזיק כתובת של ה double indirect block 1 בלוק + 2 בלוקים + 988 בלוקים
- ועוד שדה נוסף אחד נועד להחזיק כתובת של ה triple indirect block 0

חשבו מה הגודל המרבי של קובץ שניתן לאכסן ב 1514 בלוקים בסה"כ(כולל נתונים ומצביעים, אבל לא כולל את ה I-NODE עצמו) .

הערה : לצורך החישוב 1 KByte = 1024 Byte , 1 MByte = 1024 KByte

א. 3028 Kbytes

[✓ ב.] 3020 Kbytes

ג. 3000 Kbytes

ד. 3008 Kbytes

988 + 512 + 10 = 1510 בלוקים => 3020 Kb

חלק ב (55 נקודות)

ענו על השאלות 7-9.

שאלה 7 (23 נקודות)

א. (11 נק')

אנא סיימו את התוכנית להלן עבור בעיית יצרן-צרכנים(יכולים להיות כמה צרכנים, אבל רק יצרן אחד). מלאו בכל שורה ריקה פעולה(פקודה רגילה אחת או פעולה אחת על אמצעי סנכרון) **אחת בלבד**. הניחו :

- הגבול העליון של המאגר הוא N והמאגר ריק בתחילה.
- המאגר מנוהל כרשימה מקושרת כאשר פונקציה Enqueue(item) מכניסה לסוף הרשימה ו Dequeue() מוציאה מראש הרשימה. שתי הפונקציות מעדכנות את המצביעים לראש וסוף הרשימה בהתאם.

אין להשתמש באמצעי הסנכרון ללא צורך ואין להוסיף אמצעי סנכרון נוספים.

```
Semaphore BufferState1= N ; //Counting semaphore
Semaphore BufferState2 = 0 ; //Counting semaphore
Semaphore mutex1 = 1; // Binary semaphore
Semaphore mutex2 = 1; // Binary semaphore


ProducerFunction(item) {
1    down(&BS1) ;
2    ~~lock~~ down (&mutex 1) ;

3    Enqueue(item);  //Non-atomic function
4    up(&mutex 1) ;
5    up(&BS 2) ;
}

ConsumerFunction() {
    down (&BS2) ;
    down (&mutex1) ;
```

אי-אפשר להחליף בין שורות
down מחשש לdeadlock. אם
נחליף בין down(&BS1) ל-down(&mutex1),
אזי ייתכן כי ננעל את mutex1 אבל
BS1=0 כי הרשימה מלאה. נתקע
ב-down(BS1) כי מחכים להמשיך מחרים
להיות consumer אך זה תקוע ב-
down(&mutex1). גם בין down(&BS2) ל-down(&mutex) אי-
אפשר להחליף כי ייתכן וניתקע בdown(&BS2)
כש producer תקוע בdown(&mutex).

בין שני up-ים ניתן להחליף.
~~המשא~~ אין סכנה בקידום ~~up(&BS1)~~ או up(&BS2) לפני
up(&mutex1) כי תהליך שישתחרר גם כך יצטרך
את ה mutex

```
item = Dequeue(); //Non-atomic function
up(&mutex1);
up(&BS1);
return item;
}
```

ציון 1

מטרות הוספת הפעולות. בפרט ציינו האם ניתן להחליף בין שתי פעולות סמוכות:

מטרת הוספת הפעולות down(&BS1), down(&BS2) היא
למנוע מצב בו producer רושם לרשימה מלאה (אין empty slots).
ומצב בו consumer קורא מרשימה ריקה, בהתאמה. BS1 מאותחל
ל-N כי יש N empty slots. BS2 מאותחל ל-0 כי יש 0 - full.
עלינו לבצע up(&BS1) בתום קריאה ו- up(&BS2) בתום כתיבה
כדי לעדכן את מס' ה full, empty ולהעיר producer או consumer
שממתינים כי אין empty slot / full slot.

מטרת ה mutex1 היא הגנה על קטע קריטי. ביצוע שני dequeue
באותו זמן אינו מוגדר. גם ביצוע dequeue ו- enqueue
באותו זמן לא מוגדר במצב בו הרשימה באורך 1 יש להשים את head, tail.

ב. (5 נק')

אילו שורות הקוד, אם בכלל, צריך להוסיף אם הפתרון מיועד לכמה צרכנים וכמה יצרנים?
ציינו את השורות שצריך להוסיף והסבירו או הסבירו למה אין צורך להוסיף.

תשובה והסבר:

אין צורך להוסיף שורות. יש מניעה הדדית בין
כל פעולות enqueue/dequeue ולכן אין סכנה.
בשורות 3-5 נמצאים רק תהליכים כך שאפשר
לבצע ~~producer/consumer~~ enqueue/dequeue בכל סדר אפשרי
- כי רק בסוף עושים up לסמפורים - אחרי שבוצעה פעולה
משחררת. לכן גם כאן אין סכנה.

ג. (7 נק׳)

בהמשך לסעיף ב׳, הניחו שפונקציה ( )Dequeue תמיד דואגת להשאיר לפחות פריט אחד במאגר. לא צריך להתייחס לשאלה מה הסיבה לזה. אם יש רק פריט אחרון במאגר, היא מחזירה NULL וזה מטופל בקוד של צרכן עצמו(שקורא ל ()ConsumerFunction) .

הציעו שכלול לפתרון שיאפשר מקביליות גדולה יותר בין יצרנים לצרכנים. לא צריך להתייחס לשינוי רמת מקביליות בתוך יצרנים ובתוך צרכנים.

תשובה והסבר :

23
(7)

אחליף את down(&mutex1) ב-down(&mutex2) ואת up(&mutex1) ב- up(&mutex2)
ב-producer. אם יש לפחות פריט אחד במאגר אין צורך
למניעה הדדית בין פעולות enqueue ל-dequeue (רק בין
enqueue ל-dequeue ו dequeue ל-dequeue).

שאלה 8 (17 נקודות)

במערכת נתונה אין מספיק זיכרון ראשי כדי להכיל את כל קבוצת העבודה של התהליכים (Working Set) ולכן מתרחש סחרור(thrashing).

א. (7 נק׳)

אם במערכת הנ״ל האלגוריתם החלפת הדפים הוא שעון(CLOCK), אז מחוג השעון זז מהר(צריך להתקדם הרבה יחסית בשביל למצוא את הדף המתאים לפינוי) או לאט? הסבירו היטב את התשובה.

תשובה והסבר :

מחוג השעון זז מהר ונאלץ להתקדם
הרבה כדי למצוא דף מתאים לפינוי. זאת
כי קבוצת העבודה גדולה מהזיכרון המוקצה
ולכן הדפים משומשים לעיתים קרובות מאוד.
כלומר לעיתים קרובות דף יהיה במצב R=1 כי הוא
ב-working set. לכן המחוג יאלץ לעבור על הרבה
דפים עד שמוצא דף R=0 (אולי אף יאלץ לבצע
סיבוב שלם ולהגיע לדף שכבר
סימן R=0).

*

ב.(5 נק׳)

איך תשפיעה הגדלת גודל הדף על התופעה מסעי א׳ ועל גודל קבוצת העבודה של תהליך?

תשובה והסבר :

ויתכן כי קבוצת העבודה תיגדל אף יותר בגלל
פרגמנטציה פנימית (במקרה הקיצוני 9 פריימים קטנים יהפכו ל-9 גדולים).
אבל יש פחות פריימים מוקצים לתהליך ולכן מחוג השעון
יכול פחות (כי לא יכול לסבוב יותר מסיבוב שלם - החסם העליון קטן יותר).
- יותר לכל

17
(8)

ג. (5 נק׳)

מערכת ההפעלה החליטה להגדיל את סביבת העבודה(כמות הזיכרון) של תהליך כדי להקטין את מספר שגיאות הדף (הסחרור). לאחר זמן מה מערכת ההפעלה נאלצת להגדיל לתהליך נוסף את סביבת העבודה ומקטינה את סביבת העבודה של התהליך הראשון. האם ייתכן שמספר שגיאות הדף הכולל במערכת יקטן (סכום שגיאות הדף של שני התהליכים יחד)?

תשובה והסבר

כן ייתכן שיקטן. כאשר משנים באופן דינאמי את
גודל הזיכרון המוקצה לתהליך, ניתן למנוע
thrashing. (כמובן לא תמיד אפשר, אם שני התהליכים וה working set גדול מאוד
זו בעיה).

ע״י הגדלת זיכרון לא יש יותר מדי Page Fault ושחרור [illegible] פחות מדי.

המשך הבחינה בעמוד הבא

שאלה 9 (15 נקודות)

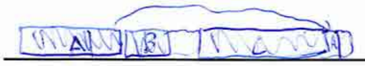
כידוע, אחת הבעיות הפוגעות בביצועי מערכת הקבצים היא קבצים שאינם שמורים באופן רציף על הדיסק הקשיח(דיסק עם חלקים מכניים).

א. (5 נק')

הוצע שיטה חדשה להקצאת מקום לקובץ - "שיטת הרווחים". השיטה הזאות משאירה מקום ריק (רווח) בגודל קבוע לאחר כל קובץ, כך שאם הקובץ צריך לגדול, הוא יכול לגדול באופן רציף. המקום השמור לאחר כל קובץ מיועד לקבוץ עצמו, אבל מותר להשתמש במקום זה עבור הקצאה לקבצים אחרים אם אין ברירה אחרת (כלומר רק במקרה שאין יותר מקום פנוי על הדיסק חוץ ממקומות השמורים לגדילה של קבצים).

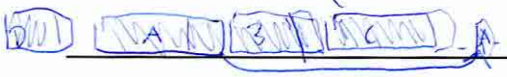
תנו 2(שתים) דוגמאות פשוטות למקרה בו, כשמשתמשים ב"שיטת הרווחים" אחד הקבצים אינו שמור באופן רציף.

תשובה והסבר :



דוגמה א': הקובץ A מילא את הרווח
שלו. שאר הדיסק הוא קבצים + הרווחים שלהם. לכן
המשך A יהיה ברווח של קובץ (לדוג' כמו ה-C)

דוגמה ב':



~~גם~~ בדוגמה זו A מילא את הרווח שלו אך הפעם יש מקום
נוסף בדיסק שאינו מוקצה לקובץ/רווח. אך בכל זאת A יגדל והקצאה
לא תהיה רציפה.

ב. (5 נק')

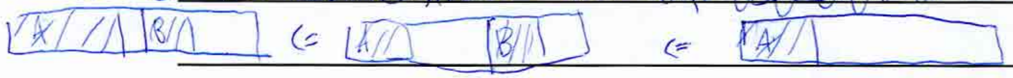
הועלתה הצעה הבאה ("שיטת החצי") : הקובץ הראשון יישמר בתחילת הדיסק. בהמשך, בכל פעם שנוצר קובץ חדש, הוא יישמר באמצע הקטע הפנוי הרצוף הגדול ביותר על הדיסק.

הסבירו מה היתרון והחיסרון של השיטה הזאות?

תשובה והסבר :

היתרון של השיטה הוא איפשור קבצים קודמים לגדול באופן רציף.

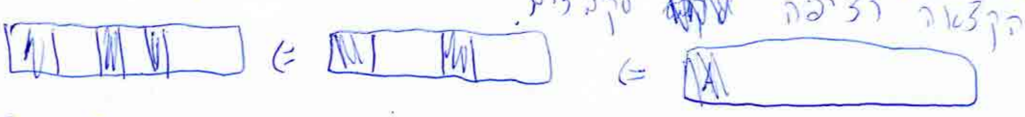
לדוגמה:



ניתן לראות כי ל-A יש מרחק לגדול.

החיסרון של השיטה היא פרגמנטציה חיצונית בדיסק, שלא מאפשר הקצאה רציפה לקבצים.

לדוגמה:



. כלומר נתקשה להקצות קובץ גדול באופן רציף.

15
(9)

ג. (5 נק')

בהנחה שמשתמשים ב"שיטת החצי" כדי למקם קבצים חדשים. איזו מבין השיטות למימוש קבצים: הקצאה רציפה (contiguous allocation), מימוש קבצים באמצעות הקצאה בשיטת רשימה מקושרת (linked list allocation), מימוש קבצים ברשימה מקושרת באמצעות אינדקס (מבוססת i-nodes) היא המתאימה ביותר לצרכנו? נמקו.

תשובה והסבר:

~~השיטה המתאימה היא הקצאה רציפה. בשיטה זו מחזיק עבור קובץ רק התחלת בדיסק ואורכו.~~

~~[illegible]~~

השיטה המתאימה ביותר היא linked list. בשיטה זו קל למצוא את המקום המתאים לתחילת קובץ. היא גמישה ומאופשרת הגדלה (בניגוד להקצאה רציפה) ובכך ממשת את היתרון המרכזי.

בשיטת ה i-node אפשר לממש זאת החל מתחילת הקובץ, אבל קשה "לרפות" זאת ~~[illegible]~~ "הקצאה הרציפה" ~~[illegible]~~

אבל אם אפשר לעשות זאת באופן יעיל ויתכן כי ה i-node הכי מתאים.

+
⇓
נדרשת
גישה אקראית
לקובץ (זה היתרון
של i-node)

המשך הבחינה בעמוד הבא

שאלון 511

88.31.2 M3

חלק ג (15 נקודות)

ענו על השאלות 10-12.

(5 נקודות לשאלה ; 15 נקודות לחלק זה).

שאלה 10

הוצע לממש את כל קריאות המערכת SYSTEM CALL כקריאה לפונקציה רגילה(רק שהיא חלק ממערכת הפעלה). האם ההצעה הזאות ניתנת למימוש עם כל הדרישות הסטנדרטיות לאופן ביצוע של קריאת מערכת? הסבירו באופן כללי איך אפשר לממש או למה לא ניתן.

5
(10)

תשובה והסבר

לא ניתן לממש זאת. ~~[illegible]~~ סכנה נגרמת מנתינת גישה לפונקציות רגישות של מערכת הפעלה קודי המשתמש ~~[illegible]~~ (וכדי שיהיה אפשרי נצטרך להפריד מ user mode ו-kernel mode).

שאלה 11

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?

ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת unshare (PID_NS) היה 20 .

ניתן להניח שכל קריאות מערכת מצליחות.

```
pid = fork();
if (pid > 0)  printf(stdout, "Grandparent's perspective on the child. PID=%d \n", pid);
else {
   unshare(PID_NS);
   pid = fork();
   if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n", pid);
   else printf (stdout, " Child's perspective on his parent: PID=%d \n", getppid());
   }
```

20

2

1

תשובה והסבר :

פלט: ומתרוללות "Grandpa... PID=20", "Parent...PID=21",

"Child...PID=1" בסדר כלשהו. תהליך הסבא אינו נמצא

ב-PID_NS כי לא מבצע את קריאת unshare, אלא רק הבן.

לכן מדפיס את הpid האמיתי (לפי ה PID_NS החיצוני) וזה 20 לפי הנתון.

הבן קורא ~~[illegible]~~ ל unshare ולכן הבן שלו (הנכד) יהיה בPID_NS החדש

אך לא הוא עצמו! לכן הבן רואה את הנכד לפי ה pid האמיתי שלו,

21, ומדפיס זאת.

אולם הנכד כתהליך ראשון ב-PID_NS ~~[illegible]~~ רואה עצמו כ-PID=1 ~~[illegible]~~

ואביו כהוא עצמו - גם PID=1. לכן מדפיס זאת.

5
(11)

10 0

שאלון 511

88.31.2 M3

מניח שהכוונה
למערכת שונה?
לא גרסאות!?

שאלה 12

5
(12)

האם במערכת הפעלה שתומכת בקונטיינרים ניתן להריץ מערכת הפעלה מסוג אחר בתוך קונטיינר נפרד?

הסבירו בזכות איזה תכונת מנגנון ה CONTAINERIZATION זה אפשרי או למה בלתי אפשרי בכלל.

תשובה והסבר:

בלתי אפשרי להריץ מערכת הפעלה מסוג אחר

(~~מ~~עם גרעין אחר, או גרסה של אותה מערכת).

כמו גרסאות linux

קונטיינר היא קלת משקל ומהירה ולא מסוגלת לתרגם
בין מערכות בעלי גרעיניות שונה הדורשת Hypervisor שמורכב יותר.

בהצלחה!